

NEXUS AI

RAG & Knowledge Engine Document

N27–N36 • Ingestion • Parsing • Chunking • Embeddings • Vector Search • Hybrid Retrieval • Reranking •
Agentic RAG • Citations • Knowledge Management

Version: 0.1 **Status:** Knowledge Engineering Foundation **Mode:** Local-first

Purpose: Define how NEXUS turns local documents and project knowledge into searchable, traceable and reusable AI context. The RAG engine is a controlled capability used by the orchestrator when evidence is needed.

SOURCES → INGEST → PARSE → CHUNK → EMBED → INDEX → RETRIEVE → RERANK → CONTEXT → GENERATE →
VERIFY → CITE

1. RAG & Knowledge Vision

NEXUS RAG is a local-first evidence retrieval system designed for private documents, project files and future knowledge sources. It should not behave as a simple PDF chatbot. Its responsibilities include ingestion, provenance preservation, retrieval, ranking, source verification and controlled context assembly.

The RAG engine sits between the knowledge layer and the orchestration layer. The orchestrator decides when retrieval is needed, while the RAG engine determines how evidence is found and returned.

Core Principles

- Local-first processing and storage.
- Source provenance must survive every transformation.
- Retrieval quality is measured, not assumed.
- Hybrid retrieval should combine semantic and lexical signals.
- Reranking improves the final evidence set.
- Agentic retrieval can rewrite and retry difficult queries.
- Citations are first-class data, not text added at the end.
- Documents and retrieved content are untrusted input and must not override system policy.
- Knowledge collections have explicit ownership and scope.
- Version changes must be traceable and re-indexable.

2. N27 — RAG System Design

The canonical NEXUS RAG pipeline is:

Source → Ingestion → Parsing → Normalization → Chunking → Metadata → Embeddings → Indexing → Query Analysis → Candidate Retrieval → Hybrid Merge → Reranking → Context Assembly → LLM → Citation Mapping → Verification.

Layer	Responsibility
Ingestion	Accept supported files and create an ingestion job.
Parsing	Extract text, structure, metadata and page/section boundaries.
Chunking	Create retrieval units with stable source references.
Embedding	Create vector representations locally.
Indexing	Persist vectors, lexical terms and metadata.
Retrieval	Return candidate evidence for a query.
Reranking	Order candidates by query relevance.
Context Builder	Select evidence within token and policy limits.
Citation Engine	Map generated claims to source spans.
Verifier	Check that the response is supported by evidence.

RAG Decision

Not every request should trigger retrieval. The orchestrator may classify a request as direct chat, memory lookup, project lookup, knowledge retrieval, or tool-driven research. Retrieval should be invoked when external/user knowledge is needed or when the task requires evidence.

3. N28 — Document Ingestion Pipeline

The ingestion pipeline converts local files into indexed knowledge while preserving the original source identity.

File Discovery → Type Detection → Security Validation → Parser → Normalizer → Metadata Extractor → Chunker → Embedding Queue → Index Writer → Validation → Ready.

Stage	Key Output
Discovery	File path, collection, source ID.
Validation	Supported type, size, permission and safety checks.
Parsing	Text plus structural elements.
Normalization	Cleaned text, normalized encoding and whitespace.
Metadata	Filename, path, page, section, timestamps, checksum, collection.
Chunking	Stable chunk IDs and source spans.
Embedding	Vector representation per chunk.
Indexing	Vector + lexical + metadata records.
Validation	Count checks, searchable status and error report.

Initial Supported Formats

PDF, DOCX, TXT, Markdown, source code, CSV/structured text and common image-containing documents where a parser/OCR adapter is available. Unsupported formats must produce a clear ingestion error rather than silent loss.

Ingestion Idempotency

Each source should have a content hash. Re-ingesting an unchanged source should avoid unnecessary duplicate indexing. Changed content creates a new document version and invalidates/rebuilds affected chunks.

4. N29 — Document Processing & Chunking Plan

Chunking must preserve enough semantic context to answer questions while remaining small enough for precise retrieval.

Chunking Concern	Design Rule
Semantic boundaries	Prefer headings, paragraphs, sections and code blocks over arbitrary splits.
Target size	Configurable token/character target based on embedding/model constraints.
Overlap	Use limited overlap only when needed to preserve boundary context.
Metadata	Attach source, page/section, version, collection and chunk position.
Code	Prefer function/class/module-aware chunks when language parsing permits.
Tables	Preserve table structure or serialize with row/column context.
Lists	Keep list headings and relevant parent context.
Long sections	Hierarchically split while retaining parent heading metadata.
Small sections	Avoid over-fragmentation; combine compatible neighboring content.

Chunk Identity

Every chunk receives a stable identifier tied to document version and location. Example concept: document_id + version_id + section/path + sequence. This allows citations and incremental re-indexing.

5. N30 — Embedding Strategy

Embeddings convert chunks and queries into vectors for semantic retrieval. NEXUS should use a local embedding model so the core knowledge workflow remains offline.

Requirement	Strategy
Provider abstraction	Embedding model is accessed through an adapter.
Local execution	Prefer local inference for privacy and offline operation.
Dimension metadata	Store vector dimension and model identity with the index.
Versioning	Embedding model/version is recorded for every indexed collection.
Batching	Batch document embeddings when resources permit.

Requirement	Strategy
Normalization	Use a consistent vector normalization policy.
Re-indexing	Embedding model changes require controlled collection rebuilds.
Query/document parity	Query embeddings must use a compatible embedding configuration.

Model Selection

The final embedding model should be selected by benchmark results on NEXUS's target documents rather than by model popularity alone. Evaluation should include retrieval recall, multilingual behavior if required, memory usage and query latency.

6. N31 — Vector Database Architecture

The vector store is responsible for semantic indexes, chunk metadata and efficient top-k retrieval. It should remain replaceable through a Vector Store interface.

Data	Required Fields
Collection	collection_id, name, scope, embedding configuration
Document	document_id, version, source metadata, checksum, status
Chunk	chunk_id, document_version, text/reference, position, metadata
Vector	embedding, dimension, embedding_model_version
Lexical index	terms/full-text representation where supported
Retrieval metadata	visibility, tags, timestamps, source path/page/section
Ingestion state	queued, processing, ready, failed, superseded

Initial Local Architecture

For the MVP, use a local vector-capable database/store that can support the chosen embedding dimension and metadata filters. The application must isolate vector operations behind an interface so the store can later be replaced with a dedicated vector database or PostgreSQL-based deployment.

7. N32 — Hybrid Search Architecture

Hybrid retrieval combines semantic similarity with lexical matching. This is important because semantic search handles meaning while lexical search is strong for exact names, error codes, filenames, APIs and technical terms.

Query → Query normalization → Semantic retrieval → Keyword/BM25-style retrieval → Candidate merge → Deduplication → Metadata filtering → Reranking.

Signal	Strength
Vector similarity	Conceptual/semantic matches.
Lexical score	Exact words, identifiers and phrases.
Metadata filters	Collection, project, document type, version, tags.
Freshness/version	Prefer current sources when policy allows.
Source authority	Optional collection/source priority.
Reranker score	Final relevance estimate.

Fusion

The initial implementation can use weighted score fusion or rank fusion. Weights should be configurable and evaluated against a fixed retrieval benchmark.

8. N33 — Reranking Strategy

Reranking is applied after candidate retrieval and before context assembly. The first-stage retrievers optimize recall; the reranker optimizes the final evidence order.

Top-K from semantic + lexical retrieval → merged candidate set → reranker → top-N evidence.

Reranking Input	Purpose
Query	User intent in normalized form.
Candidate text	Potential evidence.
Metadata	Source, section, document version and scope.
Optional task type	Coding, research, policy or general knowledge.
Optional conversation context	Resolve ambiguous references.

Reranker Policy

Reranking is optional for low-latency simple retrieval but recommended for research, multi-document and high-precision workflows. NEXUS should record reranker model/version and latency in retrieval telemetry.

Failure

If reranking fails, NEXUS may fall back to fused retrieval ranking, provided the task policy allows it. The system must not pretend reranking occurred.

9. N34 — Agentic RAG Architecture

Agentic RAG allows the system to inspect retrieval quality and adapt the search strategy instead of accepting the first result set.

Question → Query Planner → Retrieve → Grade Evidence → Sufficient? → Context/Answer OR Rewrite Query → Retrieve Again → Final Evidence → Verify.

Agentic Step	Action
Query analysis	Identify entities, intent, constraints and likely source scope.
Initial retrieval	Run hybrid search.
Evidence grading	Assess relevance and coverage.
Query rewrite	Add missing terms, reformulate ambiguous query or decompose multi-hop request.
Second retrieval	Search again using revised strategy.
Evidence synthesis	Combine complementary sources while removing duplicates.
Answer verification	Check claims against selected evidence.
Stop condition	Finish when evidence threshold or retry budget is reached.

Agentic Limits

Maximum retrieval iterations, maximum candidate count, maximum latency and maximum token budget must be configured. Agentic RAG must never enter an infinite search loop.

10. Multi-Hop Retrieval

For complex questions, NEXUS may decompose a request into sub-queries. Example: identify project → retrieve timeline → retrieve incident report → retrieve owner/team document → synthesize evidence.

Each sub-query retains a parent task ID and evidence links so the final answer can show which sources support which conclusion.

11. N35 — Citation & Source Verification System

Citations are generated from source metadata already attached to retrieved chunks. The citation system must never invent page numbers, filenames or source references.

Citation Field	Example Meaning
Document	Source document name or stable title.
Version	Document version used for the answer.
Page	Page number where available.

Citation Field	Example Meaning
Section	Heading/path where available.
Chunk	Internal evidence identifier.
Source path	Local path or safe display reference.
Quote/span	Optional short evidence span for verification.
Retrieval timestamp	When evidence was retrieved.

Citation Verification

For each important claim, the verifier checks whether the selected evidence supports it. Unsupported claims are marked uncertain, revised, or removed depending on task policy.

Citation UX

The UI should show readable source cards such as document → page/section → evidence. Internal chunk IDs may be hidden behind an advanced details view.

12. Knowledge Grounding Policy

When a task is explicitly knowledge-grounded, NEXUS should prefer evidence-backed answers and clearly distinguish retrieved facts from model inference. If evidence is insufficient, the assistant should say so rather than fabricate a confident answer.

Grounding States

GROUNDED — sufficient evidence.
 PARTIALLY GROUNDED — some claims supported, others uncertain.
 UNGROUNDED — no sufficient evidence.
 CONFLICTED — reliable sources disagree.

13. N36 — Knowledge Base Management Plan

Knowledge is organized into collections. Collections provide boundaries for retrieval, permissions and lifecycle management.

Object	Purpose
Knowledge Base	Top-level local knowledge environment.
Collection	Logical scope such as Project, Research, Personal Docs or Codebase.
Document	A source file and its metadata.
Document Version	A specific content snapshot.
Chunk	Searchable retrieval unit.
Tag	User/system classification metadata.
Index	Vector + lexical representation for a collection/version.
Ingestion Job	Track processing status and errors.

Collection Examples

SENTINEL-X Project • NEXUS Project • Academic Research • Coding Projects • Personal Notes • Technical Documentation.

Permissions

A retrieval query must be filtered by the user's allowed collections/projects before ranking. Retrieval must never use a document merely because it is semantically relevant if the caller is not authorized to access it.

14. Document Lifecycle

Discovered → Ingesting → Indexed → Active → Updated → Superseded → Archived → Deleted.

Deletion must remove or invalidate derived chunks/vectors according to the configured retention policy. A deleted source must not remain retrievable through stale indexes.

15. Versioning & Incremental Indexing

Each document is content-addressed using a checksum. When content changes, NEXUS creates a new version and indexes only changed content where possible. The active version is selected by collection policy.

Change	Action
Metadata-only	Update metadata without re-embedding if content is unchanged.
Text changed	Re-parse, re-chunk and re-embed affected document.
Embedding model changed	Re-index affected collection.
Chunking strategy changed	Re-chunk and re-index affected collection.
Document deleted	Remove/invalidate derived search records.
Collection policy changed	Rebuild metadata visibility/index filters if required.

16. Knowledge Quality & Evaluation

RAG quality must be tested independently from LLM answer quality.

Metric	Purpose
Recall@K	Whether relevant evidence appears in retrieved candidates.
Precision@K	How much retrieved evidence is relevant.
MRR / rank quality	Whether the best evidence appears early.
Citation accuracy	Whether cited sources support claims.
Groundedness	Whether answer content is supported by evidence.
Context utilization	Whether relevant retrieved information is actually used.
Ingestion accuracy	Whether source text/structure was extracted correctly.
Latency	Retrieval and end-to-end knowledge response speed.

Golden Retrieval Dataset

Create a small labeled dataset of NEXUS-specific questions with expected source documents/chunks. Use it for regression testing whenever chunking, embedding, retrieval, reranking or prompts change.

17. Prompt Injection & Untrusted Knowledge

Documents can contain instructions that look like system messages. NEXUS must treat all retrieved document text as untrusted evidence, not as executable instructions.

- Retrieved content cannot change tool permissions.
- Retrieved content cannot override system policy.
- Tool outputs cannot redefine agent roles.
- Document instructions are evidence unless the user explicitly asks to follow them as content.
- High-risk actions require policy checks independent of retrieved text.

18. Performance & Resource Strategy

Operation	Optimization
Ingestion	Batch parsing/embedding and incremental indexing.
Embedding	Cache embeddings and batch requests.
Retrieval	Metadata prefilters and appropriate top-K.
Reranking	Keep candidate set bounded.
Context	Use compact evidence and deduplicate chunks.
Storage	Avoid duplicate source/index records.
Agentic RAG	Use strict retry and latency budgets.

Operation	Optimization
Offline runtime	Respect RAM/VRAM pressure and avoid unnecessary model loads.

19. RAG Failure & Recovery

Failure	Recovery
Parser failure	Mark ingestion failed and preserve diagnostic details.
Embedding failure	Retry bounded times; then queue for later.
Vector store failure	Fallback only if another configured retrieval path exists.
No relevant evidence	Return insufficient-evidence state.
Conflicting sources	Expose conflict and source versions.
Reranker unavailable	Use fused retrieval ranking if permitted.
Agentic loop exhausted	Stop and report insufficient evidence.
Stale index	Detect version mismatch and trigger re-index.

20. RAG API / Internal Contract Concepts

The implementation should expose normalized internal contracts such as: `ingest(source)`, `index(document_version)`, `search(query, filters, top_k)`, `rerank(query, candidates)`, `build_context(candidates, budget)`, `cite(evidence)`, `verify(answer, evidence)`, and `manage_collection(operation)`.

These are architectural contracts, not a final programming-language API. N09 and implementation documents will define concrete interfaces.

21. RAG Security Requirements

- Collection-level access filtering occurs before retrieval results are exposed.
- Local file paths are validated and normalized during ingestion.
- Sensitive source metadata is not unnecessarily exposed in prompts or logs.
- Deleted sources are removed from retrieval indexes.
- Tool and document content remain untrusted.
- Indexes are stored under protected application data locations.
- Import jobs are resource-limited to avoid uncontrolled disk/RAM consumption.

22. Implementation Order for N27–N36

1. Knowledge collection/data model → 2. document ingestion → 3. parsing/normalization → 4. chunking → 5. local embeddings → 6. vector store → 7. metadata filters → 8. hybrid retrieval → 9. reranking → 10. citations → 11. RAG evaluation → 12. agentic retrieval → 13. lifecycle/version management.

The MVP should first prove a deterministic ingestion → retrieval → cited answer loop before adding autonomous query rewriting and multi-hop research.

23. MVP RAG Acceptance Gate

- A supported PDF/DOCX/text/code source can be imported locally.
- The source can be parsed and indexed with stable provenance.
- A query retrieves relevant chunks.
- The system can return a grounded answer with source information.
- The same unchanged document does not create duplicate versions on repeated ingestion.
- Unauthorized collections cannot be retrieved.

- The system reports insufficient evidence instead of fabricating a source.
- Core retrieval works without internet.
- Retrieval metrics can be measured on a small golden dataset.

24. Final RAG & Knowledge Engine Statement

NEXUS AI's knowledge engine is designed as an evidence system rather than a document chatbot. It preserves source identity from ingestion through citation, combines semantic and lexical retrieval, supports reranking and bounded agentic search, and gives the orchestrator a controlled evidence layer for reasoning. The architecture is local-first, measurable, versioned and extensible.

Document: N27–N36 RAG & Knowledge Engine Foundation • **Version:** 0.1 • **Status:** Ready for detailed component implementation